

Session 3 · Deck 3.A1

What Does It Mean to Solve a Model?

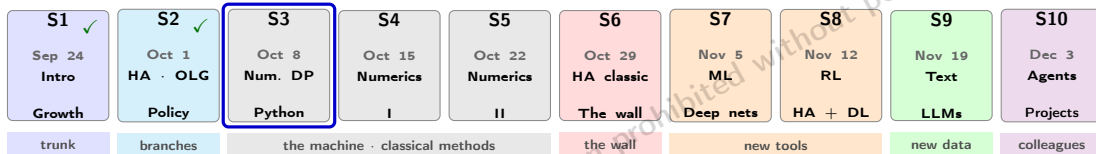
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.A1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 3.A1 what it means to solve a model · **3.A2** value function iteration on a grid · **3.A3** Euler equations and time iteration · **3.A4** the worked example, end to end.

Hands-on, 3.B5:

1 Version 0: grid VFI against the closed form

2 Data → parameters: $g, n, \alpha, \delta, \beta$

3 The calibrated economy, solved and checked

4 Test the model: untar-geted ratio, Euler residual

We Proved It Exists. Now Produce It.

Two weeks of theorems, and every one of them is an **existence** statement:

- T is a contraction, so a unique v^* exists and $T^n v_0 \rightarrow v^*$.
- A unique invariant λ^* exists, and $\lambda_0 Q^n \Rightarrow \lambda^*$ from any start.
- Capital supply and demand cross, so an r^* exists.

Not one of them is a **number**. None tells you what $v^*(0.3)$ is, how unequal λ^* is, or whether r^* is 2% or 5%.

Today: what stands between the theorem and the number — and what you must learn in order to cross it.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

We are about to ask a machine for a fixed point in an infinite-dimensional function space. It cannot represent one tenth.

The Unknown Is a Function

In a static problem you solve for numbers. Here the Bellman equation

$$v(k, z) = \max_{0 \leq k' \leq z f(k)} \{u(z f(k) - k') + \beta \mathbb{E}[v(k', z') | z]\}$$

is one equation for an **entire function** v . Bellman's move was to replace an infinite sequence of choices with a **functional equation** — a fixed point in a function space.

Same on the Euler road. Substituting $k' = g(k)$ into $u'(c) = \beta u'(c')R$ gives

$$u'(z f(k) - g(k)) = \beta u'(z f(g(k)) - g(g(k)))R(g(k)),$$

where the unknown g appears *composed with itself*.

The gap, stated once

v and g live in an infinite-dimensional space. Your computer holds finitely many numbers, each of them approximate.

So every method in this course does the same two things: pick a **finite description** of a function, and pick a **finite procedure** that drives the residual to zero.

Bellman (1957); the composed-policy form follows Judd (1998), ch. 12.

Two Roads — and Both Are Constructive

	Road 1: dynamic programming	Road 2: Euler equations
Unknown	the value function v	the policy function g
Iteration	$v_{n+1} = Tv_n$	$g_{n+1} = Kg_n$ (one root-find per point)
Why it converges	T is a contraction (Blackwell)	K is monotone and bounded
Needs	only the Bellman equation	differentiability, interiority

The theorems of 1.B2 are not decoration. A contraction argument is an **algorithm with an error bound attached**; a monotonicity argument is an algorithm with a convergence guarantee. Both roads are constructive — that is precisely why we could prove existence in the first place.

Road 2 is Coleman (1990) and Greenwood–Huffman (1995); we build it in 3.A3.

Six Decisions Before You Can Type Anything

	Decision	Question it answers	Today's crude answer
1	Represent the function	How is v stored in finite memory?	values on a grid
2	Discretize the state	Which points, how many, spaced how?	uniform grid
3	Evaluate off the grid	What is $v(k')$ between two points?	linear interpolation
4	Handle the choice	How is the max (or the root) taken?	search the grid
5	Take the expectation	How does $\mathbb{E}$ become a finite sum?	a <i>given</i> Markov chain
6	Iterate and stop	When is it converged, and how wrong is it?	$\ v_{n+1} - v_n\ < \varepsilon$

This is the list. **Every numerical method in this course is one choice from each row** — and the whole of Sessions 4 and 5 is about replacing the right-hand column with something better.

Three Sources of Error — and a Fourth

Replacing the functional equation by something finite introduces error in three places at once:

1. **State discretization.** v is known at N points; everywhere else it is a guess.
2. **Expectation approximation.** A continuous shock becomes a finite chain; $\int$ becomes $\sum$.
3. **Choice-set restriction.** The maximizer is searched over a finite set, not over the interval $[0, z f(k)]$.

Each is a modelling decision you made, and each can be measured.

The fourth one is not in the theory

Arithmetic. Every one of those finite operations is carried out in floating point. The numbers your machine adds are not the numbers you wrote down.

It is the only error source you cannot reduce by thinking harder about economics — so it is worth learning once, now, for the rest of your career.

Floating Point: The Format, and the Largest Number

A 64-bit double has three fields:

1. **Sign** S : 1 bit (0 for +, 1 for -).
2. **Exponent** E : 11 bits, read with a **bias** of 1023, so one unsigned field encodes both large and small exponents.
3. **Mantissa** M : 52 bits, the fractional part.

Normalized value ($1 \leq E \leq 2046$):

$$\text{value} = (-1)^S \times (1 + M) \times 2^{E-1023}.$$

The leading 1 is *implicit* — not stored, so it costs no bit. The two remaining exponent fields are reserved: $E = 0$ for zero and the denormals, $E = 2047$ for *inf* and *nan*.

The maximum representable value needs $E = 2046$, the largest normalized exponent, and M all ones:

$$\text{val}_{\max} = \underbrace{(2 - 2^{-52})}_{\substack{\text{binary } 1.11\dots1 \\ \approx 2}} \times 2^{2046-1023}$$

$$\approx 2 \times 2^{1023} = 2^{1024} \approx 1.80 \times 10^{308}.$$

$(2 - 2^{-52})$ is the implicit 1 plus the sum of all 52 fractional bits.

The exponent buys **range**; only the mantissa buys **precision**.

Floating Point: Converting Bits to Value

0 10000000001 101000

1. **Sign** S : $0 \rightarrow$ positive.
2. **Exponent** E : $10000000001_2 = 1025$. True exponent $= 1025 - 1023 = 2$.
3. **Mantissa** M : $101\dots$ is $1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$. Fraction $= 0.5 + 0.125 = 0.625$; significand $= 1 + 0.625 = 1.625$.
4. **Result**:

$$\text{value} = +1.625 \times 2^2 = 1.625 \times 4 = \mathbf{6.5}.$$

The 64-bit pattern was read from memory with Python's struct.

Why 6.5 is exact and 0.1 is not

$6.5 = 110.1_2$: the binary expansion **ends**, so 52 bits hold it exactly.

0.1 = 0.000110011001...₂ **never ends**. It is cut off after 52 bits and rounded, so the double called 0.1 is really 0.10000000000000000555... — the root of the 0.30000000000000004 that opened this deck.

Floating Point: Machine Epsilon

The step between neighbouring numbers is set by the **last bit** of the mantissa, scaled by the exponent. Look at it at 1.0:

$$0 \text{ } 0111111111 \text{ } 00 (-1)^0 \times (\underline{1.0}) \times 2^{1023-1023} = \mathbf{1.0}$$

[illegible]

Machine epsilon is that gap — the base precision:

$$\epsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}.$$

Scaling. At larger magnitudes the gap is multiplied by 2^{E-1023} .

- At 2^{20} (about a million) it is $2^{-52} \times 2^{20} = 2^{-32} \approx 2.3 \times 10^{-10}$.
- At 2^{52} it is 1.0: beyond $2^{53} \approx 9 \times 10^{15}$ not every integer exists, and $2^{53} + 1$ rounds back to 2^{53} .

Floating Point: Dense Near Zero, Sparse Far Out

Floating-point numbers are **dense** near zero and **sparse** further out. Each time the exponent rises by one, the distance between neighbours **doubles**: between 2^j and 2^{j+1} they sit $2^j \epsilon_{\text{mach}}$ apart.

	ϵ_{mach}	digits
float64	$2^{-52} \approx 2.2 \times 10^{-16}$	≈ 16
float32	$2^{-23} \approx 1.2 \times 10^{-7}$	≈ 7



The “floating” point

The binary point floats to keep the **relative** precision fixed, but the **absolute** error grows with magnitude. There is no fixed number of decimal places — only a fixed number of significant digits, wherever on the line you stand.

The number line is drawn with eight steps per power of two for legibility — float64 has 2^{52} .

Floating Point: Denormals and the Four Boundaries

The gap at zero. Normalized numbers stop at $2^{-1022} \approx 2.23 \times 10^{-308}$; below it, the implicit leading 1 makes anything smaller impossible.

Denormals ($E = 0$) drop the implicit 1 and fix the exponent at -1022 ,

$$\text{value} = (-1)^S \times (0 + M) \times 2^{-1022},$$

filling the gap down to zero. The smallest positive double is

$$2^{-52} \times 2^{-1022} = 2^{-1074} \approx 4.94 \times 10^{-324}.$$

Boundary	Value	Concept
max value	$\approx 1.80 \times 10^{308}$	range limit
machine epsilon	$\approx 2.22 \times 10^{-16}$	precision at 1
min normalized	$\approx 2.23 \times 10^{-308}$	full-precision floor
true minimum	$\approx 4.94 \times 10^{-324}$	absolute floor

Denormals buy range by **surrendering precision**: every leading zero in M is one significant bit lost, until 2^{-1074} has a single bit left.

What That Does to a Macro Model

Macro models routinely hold quantities that differ by fifteen orders of magnitude in the same array: K and Y around 10^{12} – 10^{14} , while r , δ and a convergence tolerance live at 10^{-2} – 10^{-8} .

At US GDP $\approx$ \$28 trillion, the distance to the next representable number is

float64	\$0.0039
float32	\$2,097,152

In single precision, **adding \$100,000 to \$28 trillion changes nothing at all**. The addition is silently discarded, and no error is raised.

Spacings computed with `numpy.spacing`; all values recorded in the session's number ledger.

The “big + small” trap

Whenever a small correction is added to a large accumulator — a market-clearing residual, a welfare sum, a capital stock — check that the small thing is not being absorbed.

Course default: float64. Machine learning defaults to float32 (or less) because gradients tolerate noise. *Accounting identities do not.* When we reach PyTorch in 3.B3, this is the first setting to check.

Three Ways the Arithmetic Lies

1. **Cancellation.** Subtracting nearly equal numbers destroys the leading digits, and what is left is noise promoted to the front:

$$(1.0 + 1\text{e-}16) - 1.0 \rightarrow 0.0$$

2. **Addition is not associative.**

$$\begin{aligned}(0.1+0.2)+0.3 &\rightarrow 0.6000000000000001 \\ 0.1+(0.2+0.3) &\rightarrow 0.6\end{aligned}$$

Why: none of the three is exact in binary, and every + rounds its exact result to the nearest double. In $0.2 + 0.3$ the storage errors cancel (exactly 0.5); in $0.1 + 0.2$ they add, the sum rounds up to 0.30000000000000004 , and the next + keeps it. The *order* changes the value.

3. **Overflow, underflow, and nan.** $10^{308} \times 10 \rightarrow \text{inf}$; $10^{-320}/10^{10} \rightarrow 0.0$; and $\text{nan} \neq \text{nan}$ is True.

Where each one finds you

Cancellation: a finite-difference derivative gets worse as the step shrinks, because the numerator is a difference of nearly equal numbers (measured in 3.B3).

Ordering: a market-clearing residual moves in the last digits when you change how you sum.

nan loses every comparison silently — one of them can make max return a wrong answer, unflagged.

Practical consequences: mark infeasible choices with a **large negative number**, not $-\infty$; compute a residual as a **ratio**, not a difference; and never test floats with ==.

Three Error Budgets, and How They Compose

Your answer differs from the truth for three independent reasons:

Budget	Comes from	Controlled by
Approximation	finitely many grid points	h , the spacing
Truncation	stopping after n iterations	ε , the tolerance
Round-off	floating-point arithmetic	the dtype

They **add**, and the largest of the three sets the total, so paying to shrink one far below the others buys you almost nothing.

A rule you can apply today

Before refining a grid, ask what your iteration tolerance already costs you. Before tightening a tolerance, ask what your grid already costs you.

In 3.A2 we will watch this happen: a grid refined from 400 to 800 points — four times the work — barely moves the value error (2.8×10^{-5} to 1.8×10^{-5}), because the iteration tolerance has already put a floor underneath it.

What “Right” Means: A Number Is a Claim

You almost never know the truth, so accuracy is reported as a **residual**: how badly does the computed policy violate an equilibrium condition it was never given?

$$\mathcal{E}(k, z) = \left| 1 - \frac{\tilde{c}(k, z)}{c(k, z)} \right|,$$

the **unit-free** consumption mistake of 1.B2. It reads directly as money: $\log_{10} \mathcal{E} = -3$ is a one-dollar error per \$1,000 of consumption.

Orders of magnitude — the unit we report in

Errors span ten orders of magnitude, so we compare them on the $\log_{10}$ scale. One unit is one **order of magnitude**, a **factor of ten**: 10^{-2} to 10^{-3} is one digit bought; 1.2 orders means the error fell $10^{1.2} \approx 16$ times.

Thresholds from Judd (1992).

ECON 282E · Session 3 · Deck 3.A1

$\log_{10} \mathcal{E}$	Verdict	Reads as
-3	acceptable	\$1 per \$1,000
-5	good	\$1 per \$100,000
-7	excellent	\$1 per \$10 million

Reporting standard for this course

State the grid, the tolerance, the dtype, and the **maximum** residual on points that were *not* used to solve — not the mean, and never a residual evaluated on the solution grid itself.

Why a Residual You Can Compute Bounds an Error You Cannot

You never observe $\|g - \hat{g}\|$: knowing it would mean knowing g . Two theorems say the things you *can* compute stand in for it.

Santos & Vigo-Aguiar (1998). With grid spacing h ,

$$\|v - v^h\| \leq \frac{M}{1-\beta} h^2, \quad \|g - g^h\| \leq \left(\frac{M}{1-\beta}\right)^{1/2} h.$$

The value converges **one order faster** than the policy — because at the optimum the first-order condition holds, so choosing wrongly by ϵ costs value only ϵ^2 .

Santos (2000). The Euler residual bounds the policy error: $\|g - \hat{g}\| \leq C \|\mathcal{E}\|$, with C set by the *curvature* of u .

The consequence people miss

A beautifully converged value function is **not** evidence of a good policy. $\|v_{n+1} - v_n\| < 10^{-6}$ is compatible with a policy wrong in the second digit — and in 3.A2 it will be.

What this licenses

The residual is *computable*; the policy error is not. Santos is the bridge that lets you report the first as evidence about the second — without it, a small residual is just a small number.

C grows as the problem flattens, so in a nearly linear model a small residual certifies **less**: an order of magnitude, not an error bar.

The Benchmark We Never Let Go Of

Set $u(c) = \ln c$, $y = zk^\alpha$ and $\delta = 1$. Then the model has a closed form:

$$g(k, z) = \alpha\beta zk^\alpha, \quad c(k, z) = (1 - \alpha\beta)zk^\alpha,$$

$$v(k, z) = A(z) + \frac{\alpha}{1 - \alpha\beta} \ln k.$$

Full depreciation is economically silly at an annual frequency. That is not the point. The point is that we know the answer, **exactly**, at every state.

The rule from 1.B1

Every numerical method in this course is tested here first. *If a method cannot recover $k' = \alpha\beta zk^\alpha$, do not trust it on anything harder.*

It is also the first place your AI assistant gets graded. A plausible-looking solver that misses a known answer by 1% is not a small problem — it is a wrong program.

Brock & Mirman (1972). Calibration used throughout Session 3: $\alpha = 0.36$, $\beta = 0.95$, $\delta = 1$; only the worked example 3.A4 adds depreciation, $\delta = 0.10$.

What Computing Teaches That the Theorem Does Not

The contraction theorem says

$\|v_n - v^*\| \leq \beta^n \|v_0 - v^*\|$ and stops there. Running the algorithm tells you things the theorem is silent about:

- **The policy converges long before the value does.** On our grid the policy stops changing at iteration **16**; the value needs **272**.
- **The error has a shape, and iterating does not remove it.** The policy error is a *sawtooth* of amplitude up to about two-thirds of a grid spacing, because the policy can only land *on* a grid point.
- **The stopping rule is not the error.** Successive iterates can differ by 10^{-6} while the answer is still $19\times$ further away.

Why this matters

Every one of these is a decision you would get wrong by reasoning from the theorem alone: how long to iterate, where to put grid points, what to report.

All three are measured in the next deck, on the model above, with the closed form as the judge.

Takeaways

- Existence theorems are not answers. The unknown is a **function**, and solving means choosing a finite description of it plus a finite procedure that kills the residual.
- Both roads out of Session 1 — Bellman and Euler — are **constructive**. The proof technique *is* the algorithm, and it carries the error bound with it.
- There are exactly **six decisions** to make. Sessions 4 and 5 are about making each of them better than we will today.
- Error comes from the grid, the tolerance, and the **arithmetic**. They add, so the smallest one is wasted effort. Use float64 for anything with an accounting identity in it.
- A computed number is a **claim**. Report the maximum unit-free residual on points you did not solve on, next to a closed form whenever one exists.

Bridge: Make Every Choice Badly, On Purpose

We now have the list of six decisions. The fastest way to understand it is to make all six in the crudest way available, run the result, and see what it costs.

- Values on a uniform grid, linear interpolation, the maximum found by searching the grid, a Markov chain handed to us, and a naive stopping rule.
- A loop of **ten lines**, and we know the exact answer it has to reproduce.

Then we count: how long it took, how wrong it is, and *which of the six crude choices* is responsible for each part of the damage.

Next (3.A2)

Value function iteration on a grid: the first complete crossing from the equation to a number.

It works — and then it tells us exactly which numerical techniques we are missing, which is the agenda for the rest of the month.